VERIQTA

# GitLab CI/CD HANDBOOK

FLAGSHIP HANDBOOK 60 PAGES

- COMPLETE CI/CD FUNDAMENTALS
- SECURE & SCALABLE PIPELINES
- DEPLOY TO ANY ENVIRONMENT
- BEST PRACTICES & REAL-WORLD EXAMPLES
- ENTERPRISE PRODUCTION READY

## From Beginner to Enterprise Production CI/CD



FASTER DELIVERY | SECURE BY DESIGN | AUTOMATE EVERYTHING | COLLABORATE BETTER

BUILD • TEST • SECURE • DEPLOY • MONITOR • REPEAT

# 1. INTRODUCTION TO GITLAB CI/CD

## WHAT CI/CD IS

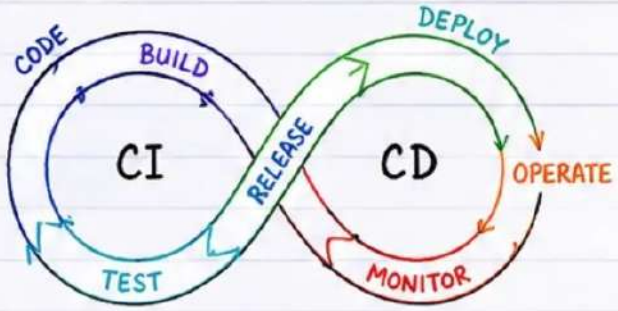
CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. It is a practice that automates the process of building, testing, and delivering code to users quickly and reliably.

> Goal: Deliver value to users faster with confidence.

## WHY GITLAB CI/CD MATTERS

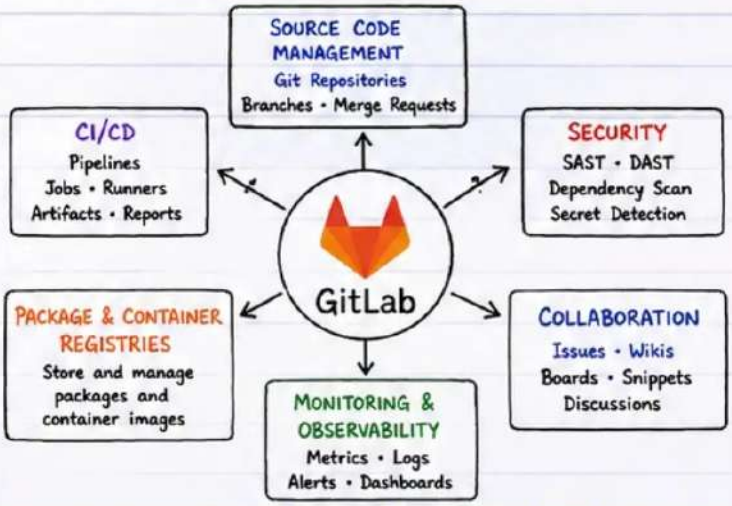
- ☑ Built-in CI/CD - no extra tools required.
- ☑ Deep integration with GitLab features.
- ☑ Secure, scalable, and highly customizable.
- ☑ Saves time through automation.
- ☑ Improve code quality and collaboration.
- ☑ Supports everything from small projects to complex enterprise systems.

## DEVOPS LIFECYCLE




PLAN → CODE → BUILD → TEST → RELEASE → DEPLOY → OPERATE → MONITOR → FEEDBACK

## GITLAB PLATFORM OVERVIEW




## HANDBOOK OBJECTIVES

- Understand GitLab CI/CD from basics to enterprise.
- Design scalable, secure, and reliable pipelines.
- Integrate testing, security, and quality into pipelines.
- Automate deployments with confidence.
- Implement best practices for production environments.
- Troubleshoot issues like a senior engineer.
- Build real-world CI/CD systems end-to-end.

## LEARNING ROADMAP

| | | |
|---|---|---|
| 1 | Fundamentals | Foundation |
| 2 | Pipeline Building Blocks | |
| 3 | Advanced Pipelines | Build & Secure |
| 4 | Security & Quality | |
| 5 | Deployments & Environments | Deliver |
| 6 | Integrations & Automation | |
| 7 | Enterprise & Best Practices | Scale & Operate |
| 8 | Troubleshooting & Optimization | |

> NOTE: This handbook takes you from beginner concepts to enterprise-grade CI/CD solutions using GitLab as the central DevOps platform.

# 2. GITLAB PLATFORM ARCHITECTURE

## GITLAB COMPONENTS

GitLab is an all-in-one DevOps platform. Its architecture is built around modular components working together to deliver the complete CI/CD experience.

| | |
|---|---|
| ① GitLab Server | The core application. |
| ② GitLab Runner | Executes jobs. |
| ③ Repository | Stores your source code. |
| ④ Registry | Stores container images and packages. |
| ⑤ Pipeline Engine | Orchestrates pipelines and jobs. |

## COMPONENT DETAILS

### GITLAB SERVER

Hosts the application, handles authentication, authorizes access, manages repositories, runs the CI/CD pipeline engine and provides APIs.

### GITLAB RUNNER

Executes CI/CD jobs. Runners can be shared, group, project or instance level. Executors can be Docker, Kubernetes, Shell, SSH, or Virtual Machine.

### REPOSITORY

Stores source code in Git. Supports branching, merging, tags, and version control.

### REGISTRY

Stores Docker images and packages securely. Integrated with CI/CD for build and deploy.

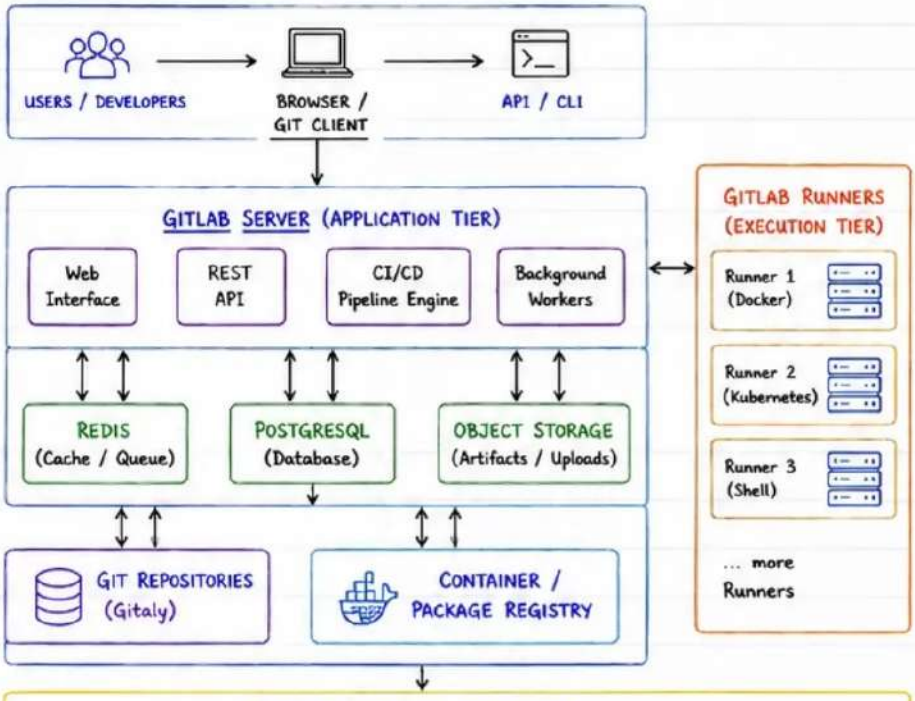
### PIPELINE ENGINE

Interprets .gitlab-ci.yml, creates pipelines, schedules jobs, manages dependencies and tracks status.

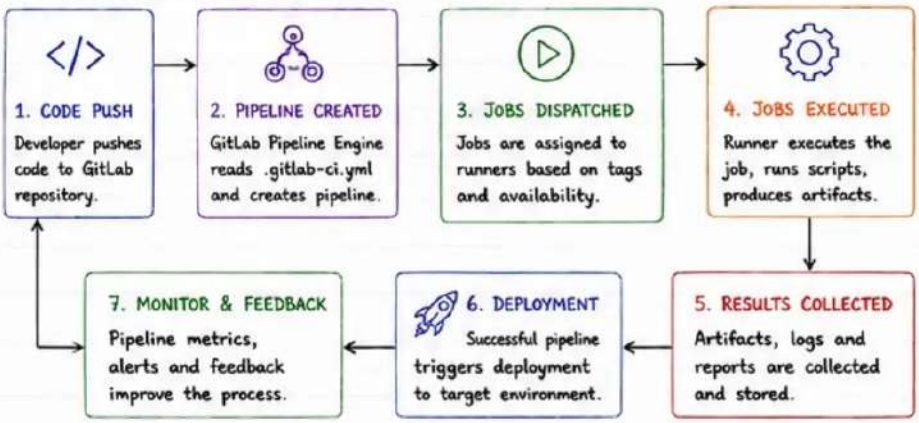
### PRODUCTION CONSIDERATIONS

- ☑ Use HA GitLab architecture.
- ☑ Separate runners from server.
- ☑ Use dedicated database & Redis.
- ☑ Backup regularly.
- ☑ Monitor performance and jobs.

## PRODUCTION ARCHITECTURE OVERVIEW




## CI/CD FLOW OVERVIEW




### KEY BENEFITS

- Single platform for the entire DevOps lifecycle.
- Tight integration between code, CI/CD, security, and deployments.
- Scalable from small teams to large enterprises.
- Secure, extensible, and cloud-native ready.

# 3. INSTALLING GITLAB

## SELF-MANAGED VS GITLAB.COM

| SELF-MANAGED GITLAB | GITLAB.COM (SaaS) |
|---|---|
| ☑ Full control over data | ☑ No infrastructure to manage |
| ☑ Custom configurations | ☑ Automatic updates |
| ☑ Works behind firewall | ☑ Global availability |
| ☑ No vendor lock-in | ☑ Faster to get started |
| ☑ Requires infrastructure | ☑ Limited customization |
| ☑ You manage updates | ☑ Dependent on GitLab |
| ☑ Best for enterprises | ☑ Best for teams/startups |

**CHOOSE SELF-MANAGED FOR:**
Compliance, security, data residency, customization, or offline environments.

## INSTALLATION METHODS

- **DOCKER** — Fast, portable, easy to manage.
- **LINUX PACKAGE** — Deb/RPM packages.
- **OMNIBUS** — All-in-one package (Linux).
- **CLOUD** — AWS, Azure, GCP Marketplace.
- **SOURCE** — For advanced customization.

**PRODUCTION NOTE:**
For production, use Omnibus (Linux) or Kubernetes Helm chart for HA deployments.

## DOCKER DEPLOYMENT

1. Install Docker

```
# On Ubuntu
sudo apt update && sudo apt install -y docker.io
```

2. Run GitLab Community Edition

```
docker run -d \
  --name gitlab \
  --hostname gitlab.local \
  --publish 80:80 --publish 443:443 --publish 2222:22 \
  --restart always \
  --volume gitlab_config:/etc/gitlab \
  --volume gitlab_logs:/var/log/gitlab \
  --volume gitlab_data:/var/opt/gitlab \
  gitlab/gitlab-ce:latest
```

3. Access GitLab

URL: http://gitlab.local
(Or use your server IP / domain)

**DOCKER VOLUMES**
Config → /etc/gitlab
Logs → /var/log/gitlab
Data → /var/opt/gitlab
Always backup /var/opt/gitlab

## LINUX INSTALLATION (OMNIBUS)

1. Add GitLab package repository

```
curl -fsSL https://packages.gitlab.com/install/repostiories/gitlab/gitlab-ce/script.deb.sh | sudo bash
```

2. Install GitLab

```
sudo apt install gitlab-ce
```

3. Reconfigure GitLab

```
sudo gitlab-ctl reconfigure
```

4. Start/Stop GitLab

```
sudo gitlab-ctl start     # start services
sudo gitlab-ctl stop      # stop services
```

**OMNIBUS INCLUDES:**
- ☑ GitLab Rails App
- ☑ PostgreSQL
- ☑ Redis
- ☑ Nginx
- ☑ Sidekiq
- ☑ Gitaly
- ☑ All pre-configured!

## INITIAL CONFIGURATION

1. Set external URL

```
sudo gitlab-ctl reconfigure
```

2. Edit configuration (if needed)

```
sudo nano /etc/gitlab/gitlab.rb
external_url 'https://gitlab.yourdomain.com'
```

3. Reconfigure

```
sudo gitlab-ctl reconfigure
```

4. Verify status

```
sudo gitlab-ctl status
```

**KEY SETTINGS**
- external_url
- nginx['listen_port']
- nginx['listen_https']
- lets_encrypt['enable']
- gitlab_rails['time_zone']
- smtp settings
- backup settings

**ENABLE HTTPS (Let's Encrypt)**

```
sudo gitlab-ctl reconfigure
# Ensure ports 80 & 443 are open
```

## FIRST LOGIN

1. Open your browser

```
https://gitlab.yourdomain.com
```

2. Default sign-in (first user)

```
Username: root
Password: Check the initial root password
   at: sudo cat /etc/gitlab/initial_root_password
```

3. Change default password

Go to: User Settings → Edit Password

**SECURITY TIP**
Change the root password immediately. Create a new admin user and use it for daily operations.

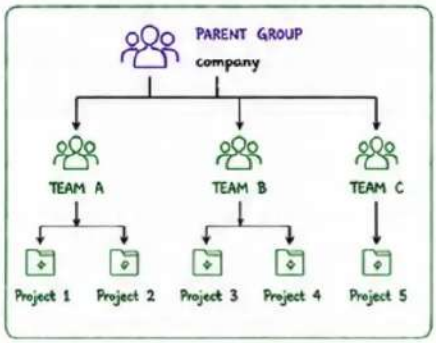
**PRODUCTION BEST PRACTICES**
- ☑ Use a dedicated server.
- ☑ Keep backups regularly.
- ☑ Configure SSL/TLS.
- ☑ Use external PostgreSQL for scale.
- ☑ Monitor resources (CPU, RAM, Disk).
- ☑ Keep GitLab updated.

GitLab is powerful, but production success comes from planning & good operations!

# 4. GitLab Projects and Repositories

## Groups

- Groups are containers for projects.
- Help organize teams and repositories.
- Control access, permissions and policies at the group level.
- Can be public, internal or private.




## Projects

A project contains your repository, CI/CD pipelines, issues, wiki, and more.

Each project belongs to a group.

Projects can be public, internal, or private.

Project settings control visibility, permissions, branches and CI/CD.

**PROJECT FEATURES**

- ☑ Repository
- ☑ CI/CD Pipelines
- ☑ Issues & Boards
- ☑ Wiki & Snippets
- ☑ Container Registry
- ☑ Packages
- ☑ Deployments
- ☑ Environments
- ☑ Access Control
- ☑ Audit Events

## Repository Structure

```
my-application/
├── .gitlab/                  # GitLab specific configs
├── .gitlab-ci.yml            # Pipeline definition
├── README.md                 # Project documentation
├── src/                      # Source code
│   ├── main.py
│   └── utils.py
├── tests/                    # Test files
│   └── test_main.py
├── Dockerfile                # Container build
├── docker-compose.yml        # Local env
├── requirements.txt          # Dependencies
└── docs/                     # Documentation
    └── architecture.md
```

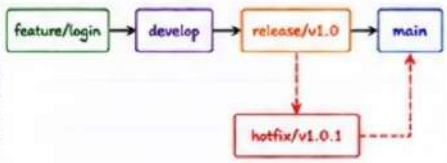
**TIP:**
A clean repo structure makes your code easy to navigate, test, and scale. Keep code, configs, docs and pipelines well organized.

## Branches

- Branches allow you to work on features, fixes, or experiments in isolation.
- Common branch types:

| BRANCH TYPE | PURPOSE |
|---|---|
| main / master | Production-ready code |
| develop | Integration branch |
| feature/* | New features |
| bugfix/* | Bug fixes |
| release/* | Release preparation |
| hotfix/* | Urgent production fixes |

**BRANCH WORKFLOW (EXAMPLE)**




## Protected Branches

- Protected branches prevent accidental deletions and unsafe pushes.
- They enforce code quality and approvals.

**COMMON PROTECTIONS**

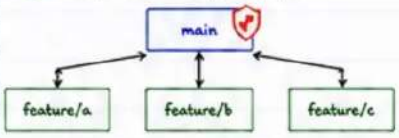
- ☑ No force push
- ☑ No branch deletion
- ☑ Require Merge Request
- ☑ Require approvals
- ☑ Require status checks (CI pipeline)
- ☑ Restrict who can push
- ☑ Signed commits (optional)

Protect your main and release branches!

## Default Branch Strategy

- The default branch is the main line of development and deployment.
- Recommended strategy:

1. Use 'main' as default branch.
2. All code changes go through Merge Requests.
3. CI/CD pipelines run on every MR and branch.
4. Only merge after approvals and successful checks.
5. Keep main always deployable.




All changes flow into main via MRs

## Repository Organization (Best Practices)

- ☑ Use groups to reflect your company structure.
- ☑ Keep projects focused and scoped to a single product or service.
- ☑ Use meaningful names for projects and repositories.
- ☑ Follow consistent branching strategy across teams.
- ☑ Protect critical branches and enforce code quality.
- ☑ Use README.md to document setup, build, and deploy.
- ☑ Use .gitlab-ci.yml at the root of the repository.
- ☑ Store secrets in CI/CD variables, not in code.

**REMEMBER:**
A well-organized repository is the foundation of a successful CI/CD pipeline and a scalable DevOps culture.

**BENEFITS**

- ☑ Easier collaboration
- ☑ Better security
- ☑ Reliable automation
- ☑ Faster delivery
- ☑ Scalable workflows
- ☑ Happy teams ☺

Plan well.
Organize better.
Ship faster.

# 5. GIT FUNDAMENTALS FOR GITLAB

☆ GIT is the foundation of everything in GitLab. Master these commands and workflows to collaborate like a pro and keep your codebase clean and safe.

### GIT CONCEPTS IN ONE LINE

Repository — Stores your project history → Staging Area — Prepares changes for commit → Commit — Snapshot of changes → Remote — Shared central repo

### ① CLONE

Clone a remote repository to your local machine.

```
git clone https://gitlab.com/group/project.git
cd project
```

### ② COMMIT

Save your changes to the local repository.

```
git add .
git commit -m "Your meaningful message"
```

### ③ PUSH

Push your committed changes to the remote repository.

```
git push origin main
```

### ④ PULL

Fetch and merge changes from the remote repository.

```
git pull origin main
```

### ⑤ FETCH

Download changes from remote, but don't merge.

```
git fetch origin
```

### ⑥ MERGE

Merge a branch into your current branch.

```
git merge feature-branch
```

### ⑦ REBASE

Reapply commits from one branch on top of another.

```
git checkout feature-branch
git rebase main
```

### ⑧ TAGS

Mark a specific point in history (release/version).

```
git tag -a v1.0.0 -m "Version 1.0.0"
git push origin v1.0.0
```

### COMMIT MESSAGE CONVENTION (EXAMPLE)

```
type(scope): short description
        |
More detailed explanation (optional)
        |
Related issue: #123
```

Types:
- feat: new feature
- fix: bug fix
- docs: documentation
- refactor: code change
- test: adding tests
- chore: maintenance

### ⑨ BRANCH WORKFLOWS (COMMON MODELS)

#### A. GITLAB FLOW (SIMPLIFIED)

main (production) → feature/* (new features) → Merge Request & Review → main

- ☑ All work in feature branches
- ☑ Merge via MR with code review
- ☑ main is always stable and deployable

#### B. GITHUB FLOW (SIMPLE PROJECTS)

main → feature/* → Merge & Deploy

- ☑ Direct merge to main
- ☑ Good for small teams
- ☑ Keep history linear

#### C. GITFLOW (ENTERPRISE)

main (production) → develop (integration) → feature/* → release/* → main; hotfix/* → main

- ☑ Separate branches for features, releases, and hotfixes
- ☑ Best for large projects
- ☑ More process, more control

### BEST PRACTICES

- ☑ Commit small, meaningful changes.
- ☑ Write clear commit messages.
- ☑ Pull before you push.
- ☑ Always work in feature branches.
- ☑ Use Merge Requests for collaboration.
- ☑ Keep main protected and stable.

### ☆ EXAMPLE WORKFLOW (END TO END)

1. Clone — git clone ...
2. Create Branch — git checkout -b feature/login
3. Commit — git add . / git commit -m "..."
4. Push — git push origin feature/login
5. Open MR — Review & Discuss
6. Merge — to main

TIP: Master these basics and everything else in GitLab CI/CD will make perfect sense!

# 6. MERGE REQUESTS

Merge Requests (MRs) are the heart of collaboration in GitLab. They provide a structured way to propose changes, review code, discuss improvements, and safely merge into target branches.

## MERGE REQUEST LIFECYCLE


## 1 CREATING MERGE REQUESTS

- Push your branch to the remote.
- In GitLab, click "New merge request".
- Choose source branch and target branch.
- Add a clear title and description.
- Assign reviewers and labels.

EXAMPLE:

Source branch : feature/login
Target branch : main
Title : Add user login with JWT
description : Implements login endpoint and token generation.

## 2 REVIEWS

- Reviewers examine code for correctness, security, performance, and standards.
- Use inline comments to suggest improvements.
- Request changes if something needs to be fixed.
- Approve when the code is good to go.

### REVIEW CHECKLIST

- ☑ Functionality works as expected
- ☑ Code is readable and maintainable
- ☑ Tests added or updated
- ☑ Security considerations reviewed
- ☑ No unnecessary changes
- ☑ Documentation updated

## 3 CODE APPROVALS

- Approvals indicate the code is ready to merge.
- Approval rules ensure quality and compliance.
- Protected branches require approvals.

### APPROVAL CONDITIONS EXAMPLE

- ☑ Minimum 2 approvals from Developers group
- ☑ No pending "Request changes"
- ☑ All pipelines must pass
- ☑ Code must be up to date with target branch

### BEST PRACTICES

- ☑ Keep MRs small and focused.
- ☑ Write clear titles and descriptions.
- ☑ Link related issues in the MR.
- ☑ Review early and often.
- ☑ Keep your branch updated with target branch.

## 4 DISCUSSIONS

- Use discussions to ask questions, share ideas, or clarify logic.
- Keep conversations respectful and productive.
- Mark as resolved when the issue is fixed.

### TIPS

- ☑ Be clear and specific.
- ☑ Reference lines of code.
- ☑ Share context and examples.
- ☑ Resolve conversations after fixes.

## 5 DRAFT MERGE REQUESTS

- Use draft MRs when work is incomplete.
- Lets the team know it's a work in progress.
- Convert to ready when it's ready for review.

### HOW TO MARK AS DRAFT

In the MR title add "[Draft]" or click "Mark as draft".

Example: [Draft] Add login endpoint

## 6 APPROVAL RULES

- Set rules to enforce code quality.
- Defined at project or group level.
- Ensures nothing is merged without checks.

### COMMON APPROVAL RULES

- ☑ Minimum number of approvals
- ☑ Specific users or groups must approve
- ☑ All discussions must be resolved
- ☑ Pipelines must succeed
- ☑ Code owners approval (if configured)
- ☑ No approvals from author (optional)

### CODE OWNERS EXAMPLE

CODEOWNERS file defines who must approve certain paths.

| | |
|---|---|
| /src/backend/ | @backend-team |
| /src/frontend/ | @frontend-team |
| /infra/ | @devops-team |

## 7 MERGE STRATEGIES

| STRATEGY | DESCRIPTION | BEST USED WHEN | PROS | CONS |
|---|---|---|---|---|
| Merge Commit (default) | Creates a merge commit to combine branches. | Preserve history of feature branches. | Easy to understand Safe, non-destructive | History can become busy with many merge commits |
| Fast-Forward Merge | Moves the target branch pointer to the source (branch must be up to date). | Linear history without extra merge commits. | Clean history Simple | Not possible if target has new commits |
| Squash and Merge | Combines all commits into one new commit. | Keep history clean and simple. | Clean history Easier to follow | Loses individual commit history |
| Rebase and Merge | Rebases source branch commits on top of target and merges. | Keep linear history with individual commits. | Clean & linear history Preserves commits | Rewriting history can be risky if shared |

### MR TITLE EXAMPLES

- ☑ Add user registration endpoint
- ☑ Fix bug in token validation
- ☑ Refactor auth service for clarity
- ☑ Update README with setup steps

### TIPS

- ☑ Use labels to categorize MRs.
- ☑ Use assignees to keep ownership clear.
- ☑ Close stale MRs to reduce noise.
- ☑ Automate checks with CI pipelines.
- ☑ Communicate, review, and merge with confidence!

# 7. GITLAB CI/CD FUNDAMENTALS

## WHAT IS GITLAB CI/CD?

GitLab CI/CD automates the build, test, package, and deploy process.
It helps teams deliver high-quality software faster and more reliably.

### KEY BENEFITS

- ☑ Automate everything
- ☑ Reduce manual errors
- ☑ Faster feedback
- ☑ Consistent delivery
- ☑ Better collaboration
- ☑ Ship with confidence

## CI/CD PIPELINE LIFECYCLE

1. Create (Push Code) → 2. Pipeline Created → 3. Jobs Scheduled → 4. Jobs Running → 5. Jobs Completed → 6. Pipeline Finished

A pipeline is triggered by events like push, merge request, schedule, or manual trigger.

## ① JOBS

- A job is the smallest unit of work in GitLab CI/CD.
- Defined in .gitlab-ci.yml.
- Runs a script on a runner.
- Example:

```
build:
  script:
    - npm install
    - npm run build
```

## ② STAGES

- Stages define the order of jobs.
- Jobs in the same stage run in parallel.
- Example:

```
stages:
  - build
  - test
  - package
  - deploy
```

## ③ RUNNERS

- Runners pick up jobs and execute them.
- Can be shared, group, or project specific.
- Supported executors: Shell, Docker, Kubernetes, SSH, Virtual Machine.

### RUNNER TYPES

| | | |
|---|---|---|
| Shared Runner | → | Managed by GitLab |
| Group Runner | → | Shared within group |
| Project Runner | → | Dedicated to project |
| Specific Runner | → | Tagged for specific jobs |

## ④ ARTIFACTS

- Artifacts are files generated by jobs and stored by GitLab.
- Useful for sharing build outputs, reports, or binaries.
- Example:

```
artifacts:
  paths:
    - dist/
    - coverage/
  expire_in: 7 days
```

### USE CASES

- ☑ Share build outputs
- ☑ Publish test reports
- ☑ Deploy files
- ☑ Store logs

## ⑤ DEPENDENCIES

- Control job execution order and data flow.
- Use needs, dependencies, or artifacts.
- Example with needs:

```
test:
  stage: test
  needs: ["build"]
  script:
    - npm test
```

### TYPES

| | | |
|---|---|---|
| needs | → | Defines job dependencies |
| dependencies | → | Download artifacts from jobs |
| artifacts | → | Pass files between jobs |

## ⑥ EXECUTION FLOW (HOW IT WORKS)

Code Push / MR / Trigger → Pipeline Created → Jobs Queued → Runner Picks Job → Job Executes → Job Complete → Pipeline Complete

### BEHIND THE SCENES

- ☑ GitLab creates a pipeline based on .gitlab-ci.yml
- ☑ Jobs are added to a queue
- ☑ Runners pick jobs, execute scripts, and return results
- ☑ Artifacts and logs are stored
- ☑ Status is updated in real-time

### IMPORTANT NOTES

- ☑ Jobs in the same stage run in parallel.
- ☑ Next stage starts when all jobs in previous stage succeed.
- ☑ Failed job stops the pipeline (unless allowed to continue).
- ☑ Artifacts and caches optimize performance.

## EXAMPLE PIPELINE OVERVIEW

| STAGE | BUILD (Parallel) | | TEST (Parallel) | | PACKAGE | | DEPLOY | |
|---|---|---|---|---|---|---|---|---|
| JOBS | install | build | unit-test | lint | docker-build | publish | staging | production |
| DEPENDS ON | - | | build | | build | | package | |
| ARTIFACTS | ✓ | | ✓ | | ✓ | | (configs) | |
| RUNNER | Docker Runner | | Docker Runner | | Docker Runner | | Kubernetes Runner | |

### QUICK REFERENCE

| | |
|---|---|
| Job | = Do work |
| Stage | = Organize order |
| Runner | = Execute jobs |
| Artifacts | = Share outputs |
| Dependencies | = Control flow |
| Pipeline | = Full execution |

# 8. UNDERSTANDING .GITLAB-CI.YML

## WHAT IS .GITLAB-CI.YML?

The .gitlab-ci.yml file is the heart of your GitLab CI/CD pipeline. It defines what runs, how it runs, and in what order.

**TIP:**
This file lives in the root of your repository and is version-controlled like your code.

## FILE STRUCTURE (HIGH LEVEL)

```
.gitlab-ci.yml
├── stages
│     - build          # Define stage order
│     - test
│     - package
│     - deploy
├── variables          # Default settings
├── default            # Default settings for jobs
├── before_script / after_script
└── jobs               # Jobs configuration
    ├── build
    ├── test
    ├── package
    └── deploy
```

## GOLDEN RULE

Every valid .gitlab-ci.yml must have at least one job and a valid stage definition (or implicit stage).

### EXAMPLE MINIMAL FILE

```yaml
stages:
  - build

build-job:
  script:
    - echo "Hello GitLab CI/CD!"
```

## 1 FILE STRUCTURE EXAMPLE

```yaml
stages:
  - build
  - test
  - package
  - deploy

variables:
  APP_NAME: my-app
  NODE_VERSION: "18"

default:
  image: node:${NODE_VERSION}
  before_script:
    - npm ci
  after_script:
    - echo "Job finished: $CI_JOB_NAME"

build:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/

unit-test:
  stage: test
  script:
    - npm run test
  needs: ["build"]

package:
  stage: package
  script:
    - echo "Packaging..."
  artifacts:
    paths:
      - build/

deploy:
  stage: deploy
  script:
    - echo "Deploying..."
  environment: production
  only:
    - main
```

## 2 KEYWORDS EXPLAINED

| KEYWORD | DESCRIPTION |
|---|---|
| stages | Defines the order of stages in the pipeline. |
| variables | Sets global variables available to all jobs. |
| default | Defines default settings for all jobs. |
| image | Docker image used to run the job. |
| before_script | Commands run before the main script. |
| script | Main commands that perform the job. |
| after_script | Commands run after the main script. |
| artifacts | Files/directories to store and pass between jobs. |
| needs | Defines job dependencies (runs after specific jobs). |
| only / except | Controls when a job runs. |
| environment | Deploy target environment (e.g., production). |
| rules | Advanced conditional job execution. |
| when | Controls when a job runs (on_success, manual, etc.). |
| tags | Assign jobs to specific runners. |
| cache | Reuse files/directories between jobs to speed up builds. |

## 3 SYNTAX BASICS

- [x] YAML format (spaces, not tabs).
- [x] Indentation matters (2 spaces recommended).
- [x] Case-sensitive.
- [x] Use colons (:) to define keys.
- [x] Use dashes (-) for lists.

### SYNTAX EXAMPLE

```yaml
job-name:
  stage: build
  image: node:18
  script:
    - npm install
    - npm run build
  artifacts:
    paths:
      - dist/
  only:
    - main
```

## 4 VALIDATION

- GitLab validates your .gitlab-ci.yml automatically.
- Errors are shown in the pipeline editor.
- Common issues:
  - ✗ Indentation mistakes
  - ✗ Missing colon
  - ✗ Invalid keyword
  - ✗ Unknown stage
  - ✗ Syntax errors

**PRO TIP:**
Use CI Lint to validate your YAML before committing.
Project → CI/CD → Editor → CI Lint

## 5 PIPELINE PARSING (HOW GITLAB READS IT)

1. Read .gitlab-ci.yml → 2. Parse YAML & Validate → 3. Build Pipeline Stages & Jobs → 4. Optimize DAG / Needs → 5. Execute Pipeline

If parsing fails → pipeline does not run.

## 7 COMMON FILE PATTERNS

| PATTERN | USE CASE | EXAMPLE |
|---|---|---|
| Single stage | Simple builds | stages: [ build ] |
| Multi stage | Build, test, deploy | stages: [ build, test, deploy ] |
| DAG (needs) | Faster pipelines | needs: [ job1, job2 ] |
| Manual jobs | On-demand actions | when: manual |
| Scheduled jobs | Nightly builds | only: [ schedules ] |

## BEST PRACTICES

- [x] Keep it simple and modular.
- [x] Reuse with YAML anchors & includes.
- [x] Use variables for values that change.
- [x] Cache dependencies to speed up builds.
- [x] Store artifacts only when necessary.
- [x] Use needs: to optimize execution time.
- [x] Keep secrets in CI/CD variables (not in code).
- [x] Document your pipeline for your team.

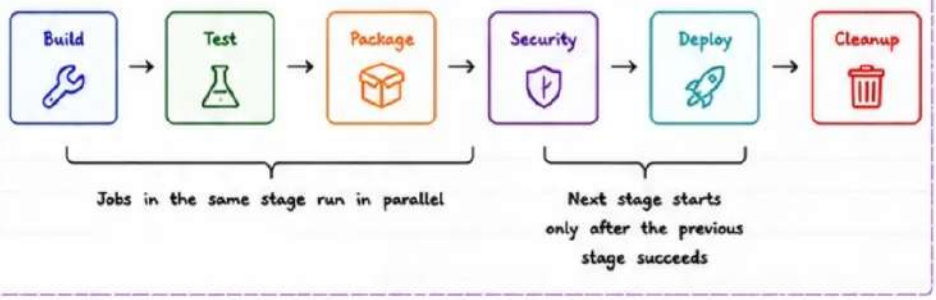
**REMEMBER**
A clean .gitlab-ci.yml leads to faster pipelines, low maintenance, and happier teams!

# 9. PIPELINE STAGES

## WHAT ARE PIPELINE STAGES?

Stages define the sequential phases of your CI/CD pipeline. Jobs within the same stage run in parallel, and stages run in the defined order from left to right.

### STAGE EXECUTION MODEL




### BEST PRACTICES

- ☑ Keep stages simple and focused.
- ☑ Fail fast: detect issues early in the pipeline.
- ☑ Run security checks before deployment.
- ☑ Deploy to smaller env first, then promote.
- ☑ Always clean up to save resources and cost.

## DEFAULT STAGE ORDER (RECOMMENDED)

1 Build → 2 Test → 3 Package → 4 Security → 5 Deploy → 6 Cleanup

## STAGES EXPLAINED

| STAGE | PURPOSE | WHAT HAPPENS | KEY JOBS EXAMPLES | OUTPUT / RESULT |
|---|---|---|---|---|
| Build | Compile source code and create build artifacts. | • Install dependencies<br>• Compile / transpile<br>• Run linters | • build<br>• compile<br>• lint | Compiled application, binaries, build artifacts |
| Test | Verify code quality and functionality. | • Unit tests<br>• Integration tests<br>• Code coverage | • unit-test<br>• integration-test<br>• coverage | Test reports, coverage results |
| Package | Create distributable artifacts. | • Package application<br>• Version artifacts<br>• Store for distribution | • package<br>• docker-build<br>• publish | Docker image, JAR, ZIP, package artifacts |
| Security | Find vulnerabilities and enforce policies. | • SAST / DAST scans<br>• Dependency scan<br>• IaC / Secret scan | • sast-scan<br>• dependency-scan<br>• secret-scan | Security reports, vulnerability status |
| Deploy | Deploy to target environment. | • Deploy to environments<br>• Run migrations<br>• Health checks | • deploy-staging<br>• deploy-production<br>• smoke-test | Running application in target environment |
| Cleanup | Remove temporary resources and artifacts. | • Remove preview envs<br>• Delete old artifacts<br>• Clean infrastructure | • cleanup-env<br>• cleanup-artifacts<br>• cleanup-cache | Clean environment, reduced cost |

## STAGE ORDERING RULES

- ☑ Stages run strictly from left to right.
- ☑ A stage starts only after the previous stage succeeds.
- ☑ Jobs within the same stage run in parallel.
- ☑ If a stage fails, downstream stages will not run.
- ☑ Use "needs" to optimize for advanced pipelines (DAG).

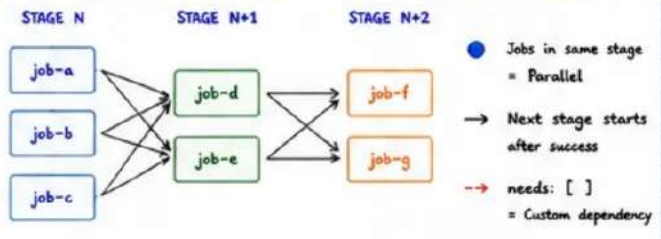
### EXAMPLE .GITLAB-CI.YML

```
stages:
  - build
  - test
  - package
  - security
  - deploy
  - cleanup
# Jobs will be defined below
# for each stage
```

### STAGE STATUS FLOW

- Success → Next stage runs
- Failed → Pipeline stops
- Skipped → Stage not executed (due to rules / conditions)
- Manual → Waiting for approval

## STAGE DEPENDENCIES




### REMEMBER

- ☑ Good stage design = Fast, reliable pipelines.
- ☑ Security before production.
- ☑ Clean up always!

### COMMON ANTI-PATTERNS

- ✗ Too many or too few stages.
- ✗ Security checks after deploy.
- ✗ Deploying without automated tests.
- ✗ Not cleaning up resources.
- ✗ Sequential jobs when parallel execution is possible.

# 10. GITLAB JOBS

## WHAT IS A JOB?

A job is a set of instructions that GitLab Runner executes in a defined environment. Jobs are the building blocks of your pipeline.

**KEY POINT**

Jobs run in the context of a stage. Multiple jobs in the same stage run in parallel.

### 1 JOB DEFINITION

- Each job is defined under the job keyword.
- It belongs to a stage.
- It contains settings and the commands to run.

### 2 SCRIPT (REQUIRED)

- The main commands executed by the job.
- Can be a single command or an array of commands.

### 3 BEFORE_SCRIPT (OPTIONAL)

- Commands executed before the script.
- Useful for setup, installing tools, logging in, etc.

### 4 AFTER_SCRIPT (OPTIONAL)

- Commands executed after the script, regardless of success or failure.
- Ideal for cleanup, notifications, or artifact handling.

### 5 RETRY

- Automatically retry a job if it fails.
- Helps reduce flaky test or network issues.

### 6 TIMEOUT

- Maximum time a job is allowed to run.
- Prevents stuck jobs from blocking pipelines.

### 7 TAGS

- Specifies which runners can pick up the job.
- Runner must have matching tags.

### 8 INTERRUPTIBLE JOBS

- Allows a running job to be canceled when a new
- pipeline starts for the same ref.
- Saves CI/CD minutes and resources.

## JOB EXAMPLE (.GITLAB-CI.YML)

```yaml
build-job:
  stage: build
  image: node:18-alpine
  tags:
    - docker
  before_script:
    - echo "Setting up environment..."
    - npm ci
  script:
    - npm run build
    - echo "Build completed!"
  after_script:
    - echo "Cleaning up..."
    - rm -rf node_modules
  artifacts:
    paths:
      - dist/
    expire_in: 7 days
  retry:
    max: 2
    when: runner_system_failure
  timeout: 30m
  interruptible: true
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: on_success
```

**EXPLAANTION**

- ☑ Runs in stage: build
- ☑ Uses Node 18 Alpine image
- ☑ Runs on Docker tagged runners
- ☑ Installs dependencies before script
- ☑ Runs the build command
- ☑ Cleans up after script
- ☑ Stores dist/ as artifact for 7 days
- ☑ Retries up to 2 times on runner issues
- ☑ Times out after 30 minutes
- ☑ Interruptible to save resources
- ☑ Runs only on main branch

## JOB PROPERTIES QUICK REFERENCE

| KEYWORD | TYPE | DESCRIPTION | EXAMPLE |
|---|---|---|---|
| script | array | Commands that define the main steps of the job. | `script:`<br>`- echo "Hello"`<br>`- make build` |
| before_script | array | Commands run before the main script. | `before_script:`<br>`- npm ci` |
| after_script | array | Commands run after the job finishes. | `after_script:`<br>`- notify` |
| retry | hash/integer | Retry job if it fails. | `retry:`<br>`max: 2`<br>`when: runner_system_failure` |
| timeout | string | Maximum time allowed for the job. | `timeout: 30m` |
| tags | array | Runner tags required to pick the job. | `tags:`<br>`- docker`<br>`- linux` |
| interruptible | boolean | Cancel job if a new pipeline is created for the same ref. | `interruptible: true` |
| artifacts | hash | Files/directories to save after the job. | `artifacts:`<br>`paths:`<br>`- dist/` |

## BEST PRACTICES

- ☑ Keep jobs small and focused.
- ☑ Use before_script for setup, not inside script.
- ☑ Save artifacts only when necessary.
- ☑ Use retry selectively, not for logical failures.
- ☑ Set reasonable timeouts to avoid hanging jobs.
- ☑ Use tags to route jobs to the right runners.
- ☑ Use interruptible for long-running, replaceable jobs.

## TYPICAL JOB EXECUTION FLOW

Runner Picks Job → before_script (Setup) → script (Main Steps) → after_script (Cleanup) → Artifacts (Upload) → Job Finished

If job fails → retries (if configured) → if still fails → pipeline fails

## REMEMBER

- ☑ Jobs run on Runners.
- ☑ Jobs in the same stage run in parallel.
- ☑ Use artifacts to pass outputs between jobs.
- ☑ Optimize jobs for speed and reliability.

# 11. VARIABLES AND SECRETS

## WHY VARIABLES?

Variables allow you to store values that can change between environments without hardcoding them in your code or pipeline.

**KEY POINT**

Never hardcode secrets like passwords, API keys, or tokens in your code or pipeline. Use variables and protect them.

## TYPES OF VARIABLES IN GITLAB CI/CD

- **CI Variables** — Defined in GitLab UI or .gitlab-ci.yml
- **Protected Variables** — Available only on protected branches tags
- **Masked Variables** — Values are hidden in job logs
- **Environment Variables** — Automatically available in job environment
- **Secrets (Secure) Management** — Store and manage secrets securely (avoid plain text)

## ① CI VARIABLES (BASIC)

- Defined at project, group, or instance level.
- Can also be defined in .gitlab-ci.yml file.

.gitlab-ci.yml example

```
variables:
  APP_ENV: "production"
  NODE_VERSION: "18"
  API_URL: "https://api.veriqta.dev"
```

## ② PROTECTED VARIABLES

- Available only when pipeline runs on protected branches or tags.
- Prevents secret exposure on untrusted refs.

Use Case:
Store production database password, deploy keys, tokens, etc. Only protected branches (main, stable/*) can access them.

## ③ MASKED VARIABLES

- Values are masked in job logs.
- GitLab hides the value even if it appears in script output.
- Masked variables must meet requirements:
  - ☑ At least 8 characters
  - ☑ No spaces
  - ☑ Allowed characters only: a-z A-Z 0-9 _@-
  - ☑ Cannot start with any of: CI_, GITLAB_, GIT_, VERIQTA_

## ④ ENVIRONMENT VARIABLES

- Provided by the runner / system automatically.
- Available in every job.
- Examples:

| VARIABLE | DESCRIPTION |
|---|---|
| CI_PROJECT_DIR | Path to the project directory |
| CI_COMMIT_SHA | The commit SHA |
| CI_COMMIT_BRANCH | The branch name |
| CI_PIPELINE_ID | The pipeline unique ID |
| CI_JOB_ID | The job unique ID |
| CI_RUNNER_TAGS | Runner tags for this job |

## ⑤ SECURE SECRET MANAGEMENT

- Use GitLab CI/CD variables (protected + masked).
- For large secrets, use external secret stores:
  - ☑ AWS Secrets Manager
  - ☑ HashiCorp Vault
  - ☑ Azure Key Vault
  - ☑ Google Secret Manager

### BEST PRACTICES

- ☑ Never commit secrets to Git.
- ☑ Use protected + masked variables.
- ☑ Limit variable scope (project/group).
- ☑ Rotate secrets periodically.
- ☑ Audit who can access variables.

## VARIABLE PRECEDENCE (HIGHEST TO LOWEST)

| | LEVEL | DESCRIPTION |
|---|---|---|
| 1 | Pipeline Trigger Variables | Variables passed when triggering pipeline manually (API, UI, trigger token) |
| 2 | Job-level Variables | Defined inside a specific job in .gitlab-ci.yml |
| 3 | Pipeline-level Variables | Defined at the top level of .gitlab-ci.yml |
| 4 | Project-level Variables | Defined in project settings |
| 5 | Group-level Variables | Defined in group settings |
| 6 | Instance-level Variables | Defined in GitLab instance (settings > CI/CD) |
| 7 | Predefined Variables | Built-in GitLab variables (CI_*, GITLAB_*, etc.) |

### EXAMPLE PRECEDENCE

If APP_ENV exists in multiple places:
Job-level value will be used.

Job-level (highest) ↑ Pipeline-level ↑ Project-level ↑ Group-level ↑ Instance-level ↑ Predefined (lowest)

## VARIABLES IN ACTION (EXAMPLE)

```yaml
variables:
  APP_ENV: "production"          # Pipeline-level
  API_URL: "https://api.veriqta.dev"

deploy-prod:
  stage: deploy
  image: alpine:latest
  variables:                     # Job-level overrides
    APP_ENV: "staging"           # This value wins
  script:
    - echo "Environment: $APP_ENV"
    - echo "API: $API_URL"
```

### TIPS

- ☑ Use variables for all environment-specific values.
- ☑ Keep secrets out of your repository.
- ☑ Use protected + masked for sensitive data.
- ☑ Document variables for your team.
- ☑ Test pipelines with safe dummy values before using real secrets.

### COMMON MISTAKES

- ✗ Hardcoding secrets in .gitlab-ci.yml
- ✗ Using unprotected variables for production secrets
- ✗ Printing secrets in logs
- ✗ Not masking tokens or passwords
- ✗ Granting variable access to everyone

### QUICK CHECKLIST

- ☑ Are all secrets stored as variables?
- ☑ Are sensitive variables protected and masked?
- ☑ Is variable scope limited?
- ☑ Are variables documented?
- ☑ Are variable values reviewed periodically?

# 12. GITLAB RUNNERS

## WHAT IS A GITLAB RUNNER?

A GitLab Runner is an application that picks up jobs from GitLab CI/CD and executes them in an isolated environment. Runners connect to GitLab, wait for jobs, run them, and report results.

## TYPES OF RUNNERS

### SHARED RUNNERS

- Provided and managed by GitLab
- Available to all projects
- Easy to get started
- Limited customization
- No access to internal resources

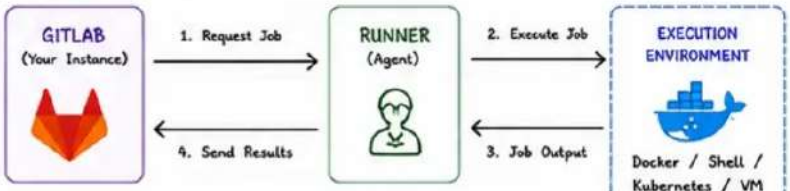
### GROUP RUNNERS

- Shared within a group
- Managed by group owners
- Access to group resources
- More control and security
- Ideal for teams/organizations

### PROJECT RUNNERS

- Dedicated to a single project
- Full control by project owners
- Access to private resources
- Highest security and isolation
- Best for critical workloads

## RUNNER ARCHITECTURE




1. The Runner polls GitLab for new jobs.
2. When a job is available, the Runner prepares the environment.
3. The job script is executed in the configured executor.
4. Results, logs, and status are sent back to GitLab.

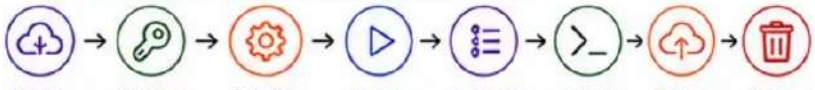
**KEY TAKEAWAY**

Runners are the workers.
GitLab is the brain.
Jobs are the tasks.

## RUNNER SCOPE COMPARISON

| TYPE | SCOPE | WHO CAN USE | MANAGED BY | BEST FOR | ACCESS LEVEL |
|---|---|---|---|---|---|
| Shared Runners | Instance-wide | All projects (on GitLab.com) | GitLab | Public projects, open source, learning | Limited<br>No internal access |
| Group Runners | Group level | Projects in the same group | Group owners | Teams, departments, internal projects | Medium<br>Access to group resources |
| Project Runners | Project level | Only that project | Project owners | Sensitive projects, private workloads, production | High<br>Full control & isolation |

## RUNNER LIFECYCLE



1. **Install** — Install GitLab Runner on a server or container.
2. **Register** — Register the runner with GitLab using the token.
3. **Configure** — Set executor, tags, and other settings.
4. **Start** — Start the runner service and connect.
5. **Pick Jobs** — Runner polls GitLab and picks available jobs.
6. **Execute** — Job is in the configured executor.
7. **Report** — Results and logs are sent back to GitLab.
8. **Cleanup** — Artifacts stored, environment cleaned up.

Runners are stateless workers. If a runner fails, another runner can pick up the job.

## RUNNER REGISTRATION

**1. Get Registration Token**

Go to: Settings > CI/CD > Runners
Copy the registration token.

**2. Install GitLab Runner**

Linux example (Ubuntu):

```
curl -L https://gitlab-runner-downloads.s3.amazonaws.com/latest/binaries/gitlab-runner-linux-amd64 -o gitlab-runner
chmod +x gitlab-runner
sudo mv gitlab-runner /usr/local/bin/
```

**3. Register the Runner**

```
sudo gitlab-runner register
--url https://gitlab.com/ # or your GitLab URL
--registration-token YOUR_TOKEN
--description "project-runner-01"
--executor docker
--tag-list "docker,linux,prod"
--run-untagged=false
--locked=false
```

**4. Start the Runner**

```
sudo gitlab-runner run
# or use systemd service
sudo systemctl enable --now gitlab-runner
```

## RUNNER EXECUTORS (COMMON)

| | |
|---|---|
| docker | Runs each job in a Docker container. Most popular and isolated. |
| shell | Runs jobs directly on the host machine. Fast but less isolated. |
| kubernetes | Runs jobs as pods in a Kubernetes cluster. Scalable and powerful. |
| virtualbox / docker+machine | Creates ephemeral VMs for each job. Good isolation, higher overhead. |

## RUNNER CONFIGURATION FILE

```
[runners]
  name = "project-runner-01"
  url = "https://gitlab.com/"
  token = "******"
  executor = "docker"
 [runners.docker]
   image = "alpine:latest"
   privileged = false
   volumes = ["/cache"]
   shm_size = 0
```

### TIPS

- ☑ Use tags to route jobs to the right runners.
- ☑ Keep runners updated.
- ☑ Use autoscaling runners for dynamic workloads.
- ☑ Secure runners and restrict network access.
- ☑ Monitor runner health and performance.

## BEST PRACTICES

- ☑ Use specific runners for production workloads.
- ☑ Tag runners clearly (e.g., docker, linux, prod).
- ☑ Protect runner machines and limit access.
- ☑ Use autoscaling for high availability.
- ☑ Clean runners regularly to avoid disk space issues.
- ☑ Avoid using shared runners for sensitive jobs.

### COMMON MISTAKES

- ✗ Using shared runners for private data.
- ✗ Not using tags, causing jobs to run anywhere.
- ✗ Leaving outdated runners without updates.
- ✗ Allowing untrusted code on privileged runners.

# 13. RUNNER EXECUTORS

## ☆ WHAT IS AN EXECUTOR?

An executor defines HOW the GitLab Runner runs your jobs. Different executors provide different isolation, performance, and use cases.

## ☆ COMMON RUNNER EXECUTORS

### 1. SHELL

- Runs jobs directly on the host machine.
- Fast and lightweight.
- No isolation between jobs.
- Best for trusted, single-tenant environments.

**Use Cases:**
Simple jobs, build scripts, internal automation.

### 2. DOCKER

- Runs each job in a Docker container.
- Good isolation.
- Easy to use.
- Requires Docker installed on the runner host.

**Use Cases:**
Most CI/CD workloads, microservices, builds, testing.

### 3. KUBERNETES

- Runs jobs as pods in a Kubernetes cluster.
- Highly scalable.
- Strong isolation.
- Best for dynamic and large-scale workloads.

**Use Cases:**
Enterprise CI/CD, large teams, cloud-native platforms.

### 4. SSH

- Executes jobs on remote machines over SSH.
- No containers by default.
- Good for legacy systems.
- Requires SSH access configuration.

**Use Cases:**
Deployments to bare metal, legacy servers, remote environments.

### 5. VIRTUAL MACHINE

- Runs jobs in full virtual machines.
- Maximum isolation.
- Heavier and slower than containers.
- Supports snapshots.

**Use Cases:**
Deployments, compliance requirements, Windows workloads.

### 6. AUTOSCALING EXECUTORS

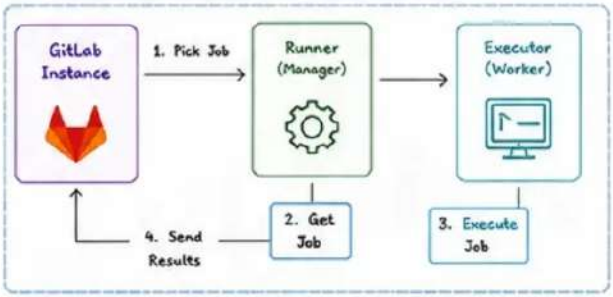
- Runners scale automatically based on job load.
- Can use Docker, Kubernetes, or VM under the hood.
- Reduces idle cost.
- Ideal for bursty workloads.

**Use Cases:**
Dynamic workloads, cost optimization, cloud environments.

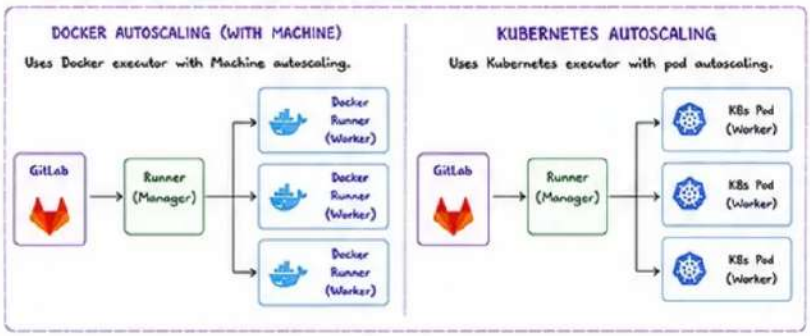
## ☆ EXECUTOR COMPARISON

| EXECUTOR | ISOLATION | PERFORMANCE | SCALABILITY | SETUP COMPLEXITY | BEST FOR |
|---|---|---|---|---|---|
| Shell | Low | Very Fast | Low | Very Easy | Trusted environments, simple jobs |
| Docker | Medium | Fast | Medium | Easy | Most CI/CD pipelines |
| Kubernetes | High | Fast | Very High | Medium/Hard | Enterprise, large scale, cloud-native |
| SSH | Low | Medium | Low | Medium | Legacy systems, remote deployments |
| Virtual Machine | Very High | Slow | Medium | Hard | High security, compliance, Windows builds |
| Autoscaling (Any) | High | Fast | Very High | Medium/Hard | Dynamic scaling, burst workloads, cloud |

## ☆ RUNNER ARCHITECTURE (WITH EXECUTOR)




## ☆ AUTOSCALING EXECUTOR EXAMPLES




## ☆ EXECUTOR SELECTION GUIDE

### HOW TO CHOOSE THE RIGHT EXECUTOR?

- ☑ Need maximum speed and trust the machine? → Shell
- ☑ Using containers and want isolation? → Docker
- ☑ Running in Kubernetes already? → Kubernetes
- ☑ Deploying to remote servers? → SSH
- ☑ Need full OS isolation or compliance? → VM
- ☑ Need automatic scaling and cost optimization? → Autoscaling

## ☆ KEY POINTS

- ☑ Executors define the environment where jobs run.
- ☑ Choose based on isolation, performance, scalability, and cost.
- ☑ Docker is the default choice for most teams.
- ☑ Kubernetes executor is best for cloud-native and scale.
- ☑ Autoscaling executors save cost by scaling on demand.
- ☑ Always monitor runner health and resource usage.
- ☑ Use tags to route jobs to the right executor.

## ☆ EXECUTOR SELECTION IN .GITLAB-CI.YML

```yaml
job-build:
  stage: build
  script:
    - echo "Building application"
  tags:
    - docker
    - linux
```

- tags: [docker, linux] → Job will run on a runner with BOTH tags.
- Use tags to target specific executors.
- Each runner can have multiple tags.

## ☆ BEST PRACTICES

- ☑ Use specific executors for different types of workloads.
- ☑ Keep runners up to date with the latest versions.
- ☑ Isolate sensitive workloads on dedicated runners.
- ☑ Use autoscaling to handle peak loads efficiently.
- ☑ Monitor, log, and alert on runner performance.
- ☑ Secure your runners and limit who can register them.

# 14. PIPELINE EXECUTION FLOW

## OVERVIEW

A GitLab CI/CD pipeline goes through a well-defined flow from creation to completion. Understanding this flow helps you troubleshoot faster and design reliable pipelines.

**KEY TAKEAWAY**

Pipelines are created by events.
Jobs run on Runners.
Status changes show real-time state.
Monitoring + logs = faster debugging.

## PIPELINE EXECUTION FLOW (HIGH LEVEL)


## DETAILED FLOW EXPLANATION

### 1 PIPELINE CREATION

- Triggered by: push, merge request, tag, schedule, manual run, API.
- .gitlab-ci.yml is read and validated.
- Pipeline object is created in GitLab.

### 2 JOB SCHEDULING

- Stages are read from .gitlab-ci.yml.
- Jobs are created and added to their respective stages.
- DAG (needs) are evaluated.
- Jobs with dependencies are ordered.

### 3 RUNNER ASSIGNMENT

- GitLab looks for available runners.
- Runner must match job tags and be online.
- Runner capabilities (executor, arch) are validated.
- Job is assigned to the selected runner.

### 4 QUEUE PROCESSING

- If no runner is available, job goes to queue.
- Job waits until a runner is free.
- Queue is FIFO per runner.
- Timeouts and concurrency limits apply.

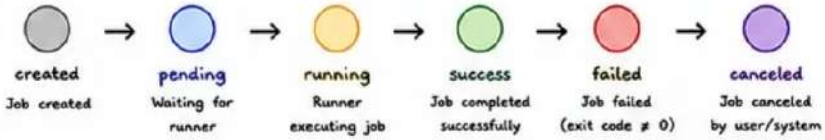
### 5 JOB EXECUTION

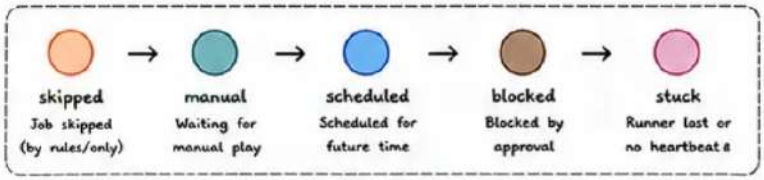
- Runner picks job from queue.
- Environment is prepared (executor starts container/VM/etc).
- before_script → script → after_script run.
- Artifacts and cache are uploaded.
- Exit code determines job status.

### 6 STATUS LIFECYCLE

- Job status is reported back to GitLab.
- Stages update in real-time.
- Pipeline status = worst status of all jobs.
- Notifications, webhooks, and approvals triggered.

## STATUS LIFECYCLE

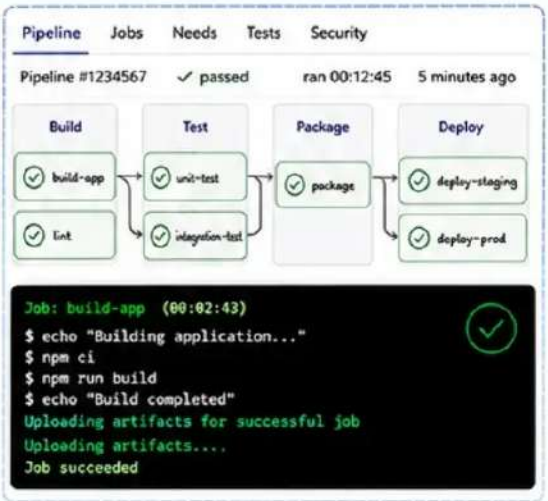


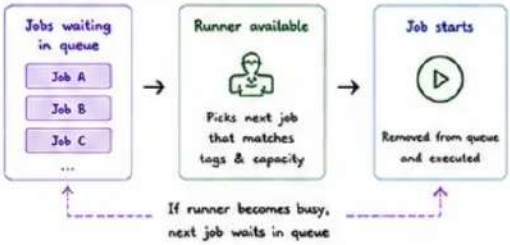

NOTE: A pipeline is successful only when all required jobs succeed.
If any job fails, the pipeline status becomes failed.

## EXECUTION MONITORING

- **Pipeline Graph** — Visual view of stages, jobs, and dependencies.
- **Job Logs** — Real-time logs from runner (stdout/stderr).
- **Job Trace** — Detailed step-by-step command output.
- **Metrics** — Track duration, success rate, runner usage, queue time.
- **Alerts & Notifications** — Slack, Email, Webhooks, PagerDuty integrations.




## QUEUING BEHAVIOR




**TIPS**

- ☑ Use tags to route jobs to the right runners.
- ☑ Keep runner concurrency balanced.
- ☑ Use needs to reduce pipeline duration.
- ☑ Monitor queue time regularly.
- ☑ Use retries for flaky jobs.

**BEST PRACTICES**

- ☑ Design small, focused jobs.
- ☑ Use caching and artifacts to optimize performance.
- ☑ Set timeouts and retry policies.
- ☑ Monitor runner health and capacity.
- ☑ Alert on long queue times or failing pipelines.

# 15. ARTIFACTS

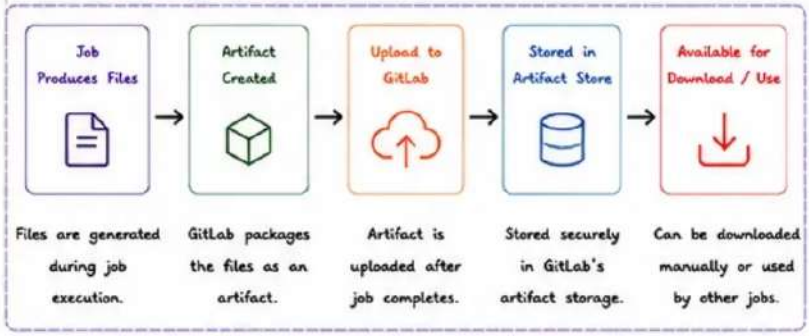
### KEY TAKEAWAY

- ☑ Artifacts persist data between jobs and stages.
- ☑ They enable traceability, sharing, and reusability.
- ☑ Use retention policies to control storage and cost.
- ☑ Keep artifacts small, relevant, and secure.

## ☆ WHAT ARE ARTIFACTS?

Artifacts are files and directories generated during a job that are uploaded to GitLab and made available for download or use in downstream jobs and stages.

## ☆ ARTIFACT UPLOAD FLOW




## ☆ ARTIFACT CREATION (EXAMPLE)

.gitlab-ci.yml

```yaml
build:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
    name: "build-$CI_COMMIT_SHORT_SHA"
    paths:
      - dist/
      - package.json
      - package-lock.json
    expire_in: 7 days
    when: always
```

Define files/ folders to keep

How long to keep artifact

Upload even if job fails (optional)

## ☆ ARTIFACT RETENTION

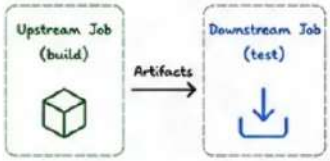
- Controlled using `expire_in` in .gitlab-ci.yml
- Default is 30 days if not specified (instance setting)
- Can be kept forever with: `expire_in: never`
- Retention applies after successful upload

| EXAMPLES | |
|---|---|
| expire_in | Retention Period |
| 1 day | Artifacts deleted after 1 day |
| 7 days | Artifacts deleted after 7 days |
| 30 days | Artifacts deleted after 30 days |
| never | Artifacts kept forever (manual cleanup) |

## ☆ ARTIFACT DOWNLOAD & USAGE

Using artifacts in downstream jobs:

```yaml
test:
  stage: test
  needs: ["build"]
  script:
  - npm test
dependencies:
  - build    # downloads artifacts from build job
```




Artifacts are automatically downloaded before the job starts.

## ☆ REPORTS (SPECIAL ARTIFACTS)

GitLab understands certain reports and shows them in the UI.

| REPORT TYPE | PURPOSE | EXAMPLE |
|---|---|---|
| JUnit | Test results | junit: report.xml |
| Coverage | Code coverage | coverage: coverage.xml |
| Code Quality | Code quality issues | codequality: gl-code-quality.json |
| SAST | Security scan results | sast: gl-sast-report.json |
| Dependency Scanning | Vulnerability in deps | dependency_scanning: gl-dependency-scanning-report.json |
| Container Scanning | Container image scan | container_scanning: gl-container-scanning-report.json |
| License Compliance | License checks | license_scanning: gl-license-scanning-report.json |

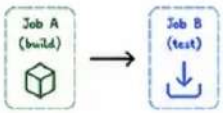
### TIP

Reports are artifacts but displayed beautifully in GitLab UI with insights and trends.

## ☆ SHARING ARTIFACTS

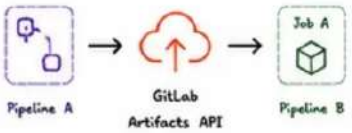
**Between Jobs (Same Pipeline)**

- Use dependencies or needs
- Ensures correct order
- Only required artifacts are downloaded




**Across Pipelines**

- Use the GitLab API to download artifacts
- Or use the "Download" button in UI
- Use project access tokens for automation




## ☆ BEST PRACTICES

- ☑ Only store what you need.
- ☑ Keep artifacts small to save storage and bandwidth.
- ☑ Set appropriate expire_in values.
- ☑ Use artifacts for build outputs, not source code.
- ☑ Never store secrets in artifacts.
- ☑ Use reports for better visibility and insights.
- ☑ Clean up expired/unused artifacts regularly.
- ☑ Use needs to reduce unnecessary downloads.

## ☆ COMMON MISTAKES

- ✗ Storing large or unnecessary files.
- ✗ Setting expire_in: never without a reason.
- ✗ Forgetting to protect sensitive data.
- ✗ Not using needs/dependencies (downloads everything).
- ✗ Not cleaning up old artifacts (storage overflow).

# 16. PIPELINE CACHING

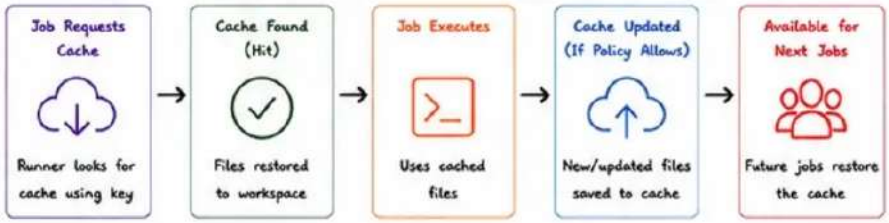
### KEY TAKEAWAY

- ☑ Caching avoids re-downloading and rebuilding dependencies.
- ☑ Good cache strategy = faster pipelines + lower CI/CD cost.
- ☑ Cache smartly, invalidate intentionally.

## WHAT IS PIPELINE CACHING?

Pipeline caching stores files and directories between jobs and pipelines so they can be reused. It reduces network usage, build time, and improves overall pipeline performance.

## CACHE FLOW




## CACHE STRATEGY

- Cache dependencies that don't change often.
- Use specific and stable keys.
- Restore cache early in the job.
- Update cache only when needed.
- Avoid caching large or unnecessary files.

## EXAMPLE: NODE.JS DEPENDENCY CACHE

.gitlab-ci.yml

```yaml
install:
  stage: install
  image: node:20
  cache:
    key: "$CI_PROJECT_NAME-node-$CI_COMMIT_REF_SLUG"
    paths:
      - node_modules/
    policy: pull-push
  script:
    - npm ci
```

- Unique key per branch
- Cache the node_modules folder
- Pull cache before job, push after job
- Install using cached deps

## CACHE KEYS

| KEY TYPE | EXAMPLE | WHEN TO USE | DESCRIPTION |
|---|---|---|---|
| Static Key | "node-cache" | Simple projects | Same key for all pipelines (not recommended) |
| Branch Key | "node-$CI_COMMIT_REF_SLUG" | Per branch | Isolates cache per branch |
| File Based | "node-$CI_PROJECT_ID-$CI_COMMIT_REF_SLUG" (with files: [package-lock.json]) | Dependencies change | Changes when lock file changes (best practice) |
| Fallback Key | key: "$CI_COMMIT_REF_SLUG" fallback_keys: - "main-node-cache" | Shared fallback | Use default cache if branch cache missing |
| Composite Key Key | "${CI_PROJECT_NAME-${CI_JOB_NAME-${CI_COMMIT_REF_SLUG}" | Complex projects | Unique per job and branch |

## CACHE POLICIES

| POLICY | BEHAVIOR | USE CASE |
|---|---|---|
| pull | Download cache if available (never upload) | Read-only caches, speed up without updating |
| push | Upload cache (ignore existing) | Publish cache for later jobs |
| pull-push | Download if exists, then upload after job | Most common policy |
| pull-if-not-exists | Download only if cache exists, do not upload | Prevent overwriting existing cache |

## DEPENDENCY CACHING EXAMPLES

| Node.js | Python | Java (Maven) | Java (Gradle) | Go | .NET |
|---|---|---|---|---|---|
| Cache node_modules/ | Cache .cache/pip/ | Cache .m2/repository/ | Cache .gradle/ | Cache $GOPATH/pkg/mod | Cache packages/ .nuget/ |
| Key based on package-lock.json | Key based on requirements.txt | Key based on pom.xml | Key based on build.gradle | Key based on go.sum | Key based on *.csproj |

## PERFORMANCE IMPROVEMENTS

- ☑ Reduces dependency download time by 60% - 90%.
- ☑ Speeds up builds, tests, and deployments.
- ☑ Less network traffic and external service load.
- ☑ Lower CI/CD minutes and infrastructure cost.
- ☑ Consistent and faster developer feedback loop.


## CACHE TROUBLESHOOTING

| ISSUE | POSSIBLE CAUSE | HOW TO FIX |
|---|---|---|
| Cache not found | Wrong key, different branch, cache expired | Check key, fallback_keys, and retention |
| Cache not uploaded | Policy set to pull or push skipped | Use pull-push or push policy |
| Large cache size | Too many files cached | Cache only necessary paths, exclude big folders |
| Stale / incorrect cache | Cache not invalidated | Use file-based keys, clear cache |
| Pipeline still slow | Cache path wrong or not restored early | Verify paths, restore in earlier stage |

## BEST PRACTICES

- ☑ Cache only what is necessary.
- ☑ Use file-based keys for automatic invalidation.
- ☑ Keep cache size small and relevant.
- ☑ Use pull-push for most use cases.
- ☑ Avoid caching build outputs—cache dependencies only.
- ☑ Regularly review and clean unused caches.

### TIP

Use lock files (like package-lock.json, pom.xml, go.sum, requirements.txt) in your cache key for best results.

### COMMON MISTAKES

- ✗ Using the same static key for all pipelines.
- ✗ Caching too much (build artifacts, logs, binaries).
- ✗ Not using fallback keys.
- ✗ Forgetting to update cache when dependencies change.
- ✗ Relying on cache without validating correctness.

# 17. PIPELINE RULES

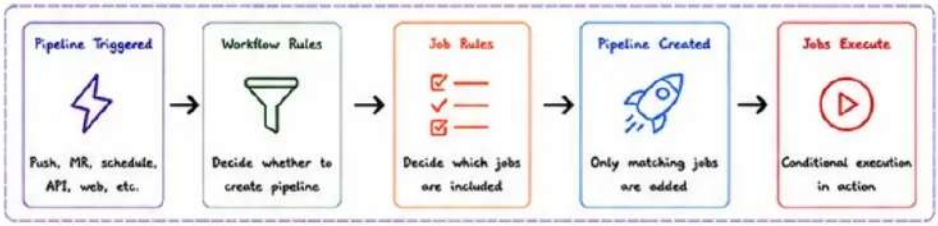
**KEY TAKEAWAY**

- ☑ Rules control WHEN jobs and pipelines run.
- ☑ Use rules for flexibility and future-proofing.
- ☑ only/except are legacy but still useful.
- ☑ Good rules = faster pipelines + lower cost.

## ☆ WHAT ARE PIPELINE RULES?

Rules define when a job is included in a pipeline. They allow conditional execution based on variables, branches, events, file changes, and more.

## ☆ PIPELINE EVALUATION FLOW




## ☆ RULE TYPES

### 1. rules (RECOMMENDED)

- Most powerful and flexible.
- Supports conditions, variables, changes, when, allow_failure, etc.
- Evaluated top-to-bottom.

### 2. only (LEGACY)

- Includes a job only in specific refs or events.
- Limited conditions.
- Avoid mixing with rules.

### 3. except (LEGACY)

- Excludes a job from specific refs or events.
- Limited conditions.
- Avoid mixing with rules.

## ☆ EXAMPLES

**A. Using rules (Recommended)**

```yaml
build-job:
  stage: build
  script: echo "Building..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: on_success
    - if: '$CI_COMMIT_TAG'
      when: on_success
    - when: never
```

Run on main branch
Run on tags
Else never run

**B. Using only (Legacy)**

```yaml
test-job:
  stage: test
  script: npm test
  only:
    - main
    - merge_requests
    - tags
```

Run only on main, MR and tags

**C. Using except (Legacy)**

```yaml
deploy-job:
  stage: deploy
  script: ./deploy.sh
  except:
    - branches
    - tags
```

Do not run on branches or tags

## ☆ WORKFLOW RULES

Control whether a pipeline is created at all.

```yaml
workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always
    - when: never
```

If none of the workflow rules match, the pipeline will NOT be created.

## ☆ CONDITIONAL EXECUTION EXAMPLES

| CONDITION | DESCRIPTION |
|---|---|
| if: '$CI_COMMIT_BRANCH == "main"' | Run on main branch |
| if: '$CI_COMMIT_TAG' | Run on tags |
| if: '$CI_PIPELINE_SOURCE == "schedule"' | Run on scheduled pipelines |
| if: '$CI_MERGE_REQUEST_ID' | Run in Merge Request pipelines |
| changes: - "src/**/*" | Run only if files changed |
| exists: - Dockerfile | Run only if file exists |
| when: manual | Manual trigger |
| when: never | Never run |

## ☆ DYNAMIC PIPELINES

Rules + variables allow dynamic behavior and multi-pipeline strategies.

### 1. Child Pipelines

Trigger child pipeline based on rules. Great for microservices or mono-repos.

### 2. Matrix Pipelines

Use variables + rules to create multiple job variations.

### 3. Environment Based

Run jobs only for specific environments (dev, staging, prod).

## ☆ PRACTICAL EXAMPLE: CONDITIONAL DEPLOY

```yaml
deploy:
  stage: deploy
  script: ./deploy.sh
  environment:
    name: production
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
      allow_failure: false
    - if: '$CI_COMMIT_TAG'
      when: manual
    - when: never
```

Run on main (manual)
Run on tags (manual)
Never in other cases

## ☆ RULES EVALUATION ORDER

1. Rules are evaluated top to bottom.
2. First matching rule is applied.
3. If no rule matches → job is excluded.
4. Avoid overlapping or conflicting rules.

**TIP**

Always end with - when: never to prevent accidental execution.

## ☆ BEST PRACTICES

- ☑ Use rules instead of only/except.
- ☑ Keep rules simple and readable.
- ☑ Use variables for flexibility.
- ☑ Avoid duplicate pipelines with workflow rules.
- ☑ Test rules using CI Lint.
- ☑ Document complex rules.

## ☆ COMMON MISTAKES

- ✕ Mixing rules with only/except.
- ✕ Forgetting the default "when".
- ✕ Overlapping rules causing unexpected runs.
- ✕ Not testing rules with CI Lint.
- ✕ Creating duplicate pipelines.

# 18. BRANCH-BASED PIPELINES

**KEY TAKEAWAY**

- ☑ Different branches have different purposes.
- ☑ Pipelines should adapt to branch type.
- ☑ Protect what matters. Automate the rest.
- ☑ Fast feedback on feature branches.
- ☑ Safe, reliable deployments from main.

## WHAT IS BRANCH-BASED PIPELINES?

Branch-based pipelines run automatically when code is pushed to specific branches. Each branch type has a defined workflow, rules, and deployment strategy.

## BRANCH OVERVIEW

**FEATURE BRANCHES**

Used for new features and changes.
Short-lived.
Created from main.
Merged back via MR.

**RELEASE BRANCHES**

Stabilization branch for a release.
Testing, QA and bug fixes before release.

**MAIN BRANCH**

Production-ready code.
Always deployable.
Source of truth.
Protected and stable.

**HOTFIX BRANCHES**

Urgent fixes for production issues.
Created from main.
Protected and quickly.

**PROTECTED BRANCHES**

Branches with rules that prevent direct pushes and enforce quality and approvals.

## PIPELINE BEHAVIOR BY BRANCH TYPE

| BRANCH TYPE | TRIGGER | PIPELINE TYPE | DEPLOYMENT | PURPOSE |
|---|---|---|---|---|
| Feature Branches | Push / MR | Branch Pipeline (Fast feedback) | No deployment (or Review App) | Build, Test, Lint, Security |
| Release Branches | Push | Branch Pipeline (Full validation) | Staging / Pre-Prod | Integration testing, QA, UAT |
| Main Branch | Push / Merge MR | Branch Pipeline (Production) | Production | Reliable production deployments |
| Hotfix Branches | Push | Branch Pipeline (Fast track) | Production (Fast deploy) | Quick fixes, minimal delay |
| Protected Branches | Push blocked | N/A | N/A | Enforce process and safety |

## PIPELINE FLOW BY BRANCH TYPE


## PROTECTED BRANCH SETTINGS (EXAMPLE)

- ☑ Require Merge Requests
- ☑ Require code owner approval
- ☑ Require status checks to pass
- ☑ Require up-to-date branch
- ☑ Restrict who can push
- ☑ Prevent force pushes
- ☑ Prevent branch deletion

**EXAMPLE PROTECTED BRANCHES**

main
release/*
production
critical/*

## EXAMPLE: .GITLAB-CI.YML RULES BY BRANCH

```yaml
# Feature branches
feature-pipeline:
  stage: test
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^feature\//'
      when: always
    - when: never

# Release branches
release-pipeline:
  stage: test
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^release\//'
      when: always

# Main branch
production-pipeline:
  stage: deploy
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: always

# Hotfix branches
hotfix-pipeline:
  stage: deploy
  rules:
    - if: '$CI_COMMIT_BRANCH =~ /^hotfix\//'
      when: always
```

Run jobs only on matching branch types

## BEST PRACTICES

- ☑ Keep feature branches short-lived.
- ☑ Use Merge Requests for all changes.
- ☑ Protect main and release branches.
- ☑ Run fast pipelines on feature branches.
- ☑ Deploy only from main or release.
- ☑ Use environments for deployments.
- ☑ Automate checks: test, lint, security.
- ☑ Monitor pipeline success and failures.

## COMMON MISTAKES

- ✗ Pushing directly to main.
- ✗ Long-lived feature branches.
- ✗ No branch protection rules.
- ✗ Deploying from feature branches.
- ✗ Skipping tests for hotfixes.
- ✗ Not cleaning up old branches.

**TIP**

A good branching model + smart pipelines = Faster delivery, higher quality, and fewer production issues.

## PIPELINE BEHAVIOR SUMMARY

# 19. MERGE REQUEST PIPELINES

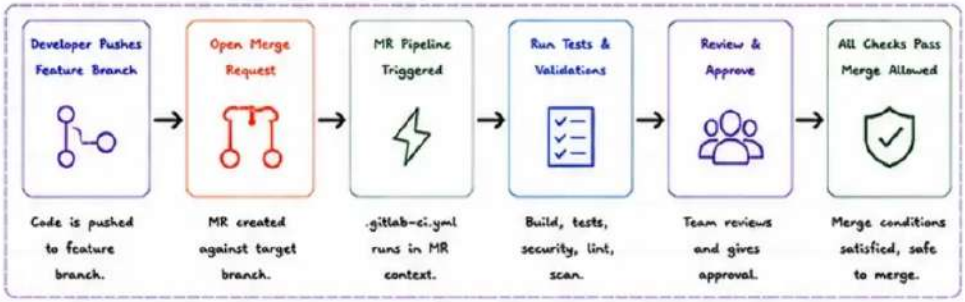
### KEY TAKEAWAY

- ☑ MR pipelines run automatically for every Merge Request.
- ☑ They validate changes before merging.
- ☑ Catch issues early and improve code quality.
- ☑ Integrate reviews, approvals, and checks.
- ☑ Safe merges lead to stable production.

## ☆ WHAT ARE MERGE REQUEST PIPELINES?

Merge Request (MR) pipelines run when a developer opens or updates a merge request. They test and validate the proposed changes without affecting the target branch until the MR is merged.

## ☆ MERGE REQUEST PIPELINE FLOW FLOW




## ☆ MR PIPELINES (CI CONTEXT)

- Runs in isolated environment.
- Uses source branch code.
- Tests changes before they enter the target branch.
- Can access target branch for comparison.
- Does not deploy to production.

## ☆ VALIDATION IN MR PIPELINES

- ☑ Code compilation / build
- ☑ Unit tests
- ☑ Integration tests
- ☑ Static code analysis (SAST)
- ☑ Security scans & dependency checks
- ☑ Code quality & linting
- ☑ Infrastructure validation (IaC)
- ☑ Docker image build (optional)
- ☑ Preview environment deployment (optional)

**TIP**

Fail fast, give clear feedback. MRs should be small, tested, and reviewable.

## ☆ EXAMPLE: .GITLAB-CI.YML (MR PIPELINE)

```yaml
stages:
  - build
  - test
  - security
  - review

variables:
  GIT_DEPTH: "50"

workflow:
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
      when: always
    - when: never

build:
  stage: build
  script:
    - echo "Building application..."
    - npm ci
    - npm run build

test:
  stage: test
  script:
    - npm test -- --coverage

lint:
  stage: test
  script:
    - npm run lint

sast:
  stage: security
  script:
    - echo "Running SAST scan..."
  allow_failure: false

review:
  stage: review
  script:
    - echo "MR pipeline completed successfully"
  rules:
    - if: '$CI_MERGE_REQUEST_ID'
      when: always
```

Pipeline runs ONLY for Merge Requests

## ☆ CODE REVIEW AUTOMATION

**Automated Test Results**
Post pipeline results as MR widgets and comments.

**Code Quality Reports**
SonarQube, CodeClimate, or built-in reports.

**Security Scan Results**
SAST/DAST findings right inside the MR.

**Inline Code Discussion**
Leave comments on specific lines of code.

## ☆ APPROVAL INTEGRATION

- Require approvals before merge.
- CODEOWNERS auto-assign reviewers.
- Protect critical paths with approval rules.
- Integrate with Slack / Email / Teams notifications.

## ☆ MERGE CHECKS (MUST PASS BEFORE MERGE)

| CHECK TYPE | DESCRIPTION |
|---|---|
| ✓ Pipeline Success | All required jobs passed. |
| ✓ Required Approvals | All mandatory approvals granted. |
| ✓ Status Checks | All status checks are successful. |
| ✓ Code Quality Gate | Quality gate passed (SonarQube, etc). |
| ✓ Security Gate | No critical/high vulnerabilities. |
| ✓ Merge Conflicts | No merge conflicts with target branch. |
| ✓ Branch Rules | Branch protection rules satisfied. |

## ☆ BEST PRACTICES

- ☑ Keep MRs small and focused.
- ☑ Run full test suite in MR pipelines.
- ☑ Use meaningful MR titles and descriptions.
- ☑ Automate as many checks as possible.
- ☑ Provide fast feedback to developers.
- ☑ Don't allow merge on failing pipelines.

## ☆ COMMON MISTAKES

- ✗ Skipping tests in MR pipelines.
- ✗ Allowing merge without approvals.
- ✗ Ignoring flaky tests.
- ✗ MRs that are too large and hard to review.
- ✗ Not using branch protection rules.

### MR PIPELINE BEHAVIOR SUMMARY

Trigger on MR + Run Validations + Review & Approve + All Checks Pass = Merge Allowed